WSE-3 Scaling Analysis: Picasso Graph Coloring on Cerebras

Picasso / LWW transport working notes

Last updated: 2026-04-24

Abstract

This document records **where each of our current kernel implementations stands against a WSE-3 deployment target**, what architectural constraints we discovered, how the layout and algorithm work around them, and what the next concrete steps look like (bigger test cases, timing plan, on-device recursion feasibility). It is a working reference: state changes, and every claim below cites an artifact under `runs/local/`.

Contents

1 A WSE-3 primer (in plain words) 1
- 1.1 What is WSE-3? . . . 1
- 1.2 What is a "wavelet"? . . . 2
- 1.3 Why do we need "colors"? . . . 2
- 1.4 Why do we need queues? . . . 3
- 1.5 Why are queues so limited? . . . 4
- 1.6 Why are colors also limited? . . . 4
- 1.7 How does our algorithm sit on this? . . . 5
- 1.8 Glossary . . . 5

2 Summary: which implementations scale to WSE-3? 7
- 2.1 Pipelined-LWW sub-variants . . . 7

3 The implementation story, how we got here, step by step 8
- 3.1 `sw-relay`, start with something that just works . . . 9
- 3.2 `pipelined-lww bidir`, the first attempt at a fabric pipeline . . . 10
- 3.3 `pipelined-lww east`, half the colors by exploiting Path C . . . 11
- 3.4 `pipelined-lww east_seg`. Break the per-PE color tax with bridges . . . 12
- 3.5 `pipelined-lww 2d` (iter 2), proving the dual-axis idea on a 2×2 toy . . . 13
- 3.6 `pipelined-lww 2d_seg`, single-axis segmented in 2D namespace . . . 14
- 3.7 `pipelined-lww 2d_seg2`. Composing everything for any grid . . . 15
- 3.8 `pipelined-lww 2d_multicast`, an orthogonal optimization . . . 18
- 3.9 `hw-filter`, the dead-end branch worth knowing about . . . 18

4 WSE-3 architectural limitations that shaped the design 19
- 4.1 Six-queue-per-PE cap (hard) . . . 19
- 4.2 Single rx-direction per color (hard on WSE-3) . . . 19
- 4.3 CP3: same-color inject + non-RAMP rx is silently dropped . . . 19
- 4.4 One OQ = one color . . . 19

4.5 Finite fabric-color budget (∼ 24 usable) . . . 19
4.6 OQ depth is small (∼ 32 wavelets), backpressures without deadlock . . . 19
4.7 Monotone ID vs. geometry mismatch . . . 20

5 How we overcame each limitation **20**
5.1 Queue-cap collapse via in-band opcodes . . . 20
5.2 Color reuse via bridges (CP2a pattern, generalized to 2D) . . . 20
5.3 Single-rx constraint → alternating-chain reduce . . . 21
5.4 CP3 prohibition → CPU-mediated re-injection . . . 21
5.5 Anti-diagonal SW delivery → per-row back-channel . . . 21
5.6 OQ-depth backpressure, acknowledged, CP2d.e pending . . . 21

6 How the algorithm fits the hardware **21**

7 Timing data collected so far (simulator, WSE-3 arch) **22**
7.1 Small-grid comparison (1D 4-PE, all 13 tests) . . . 22
7.2 Dual-axis 2d_seg2 timings (new, 2026-04-24) . . . 22
7.3 What we are still missing . . . 23

8 Plan **23**
8.1 Bigger simulator test cases . . . 23
8.2 CS-3 hardware runs (real WSE-3) . . . 24
8.3 CP2d.e, dense-graph scaling fix . . . 24
8.4 On-device recursion feasibility . . . 24
8.5 Diagnose the 4×4 / 2×2 regression-mode flake . . . 25

9 Open questions **25**

10 File and run-artifact index **25**

1 A WSE-3 primer (in plain words)

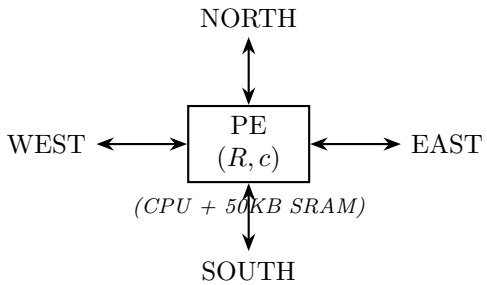
Before we get into the kernels, let us walk through what WSE-3 actually is and why it forces us to write code the way we do. If you already know the chip, you can skim. If you do not, every later section will read more clearly after this.

1.1 What is WSE-3?

WSE-3 is one big piece of silicon. It is roughly the size of a dinner plate. Most chip companies cut a wafer into thousands of small chips. Cerebras does not. They keep the wafer whole and treat it as one giant chip. The result is about **900,000 tiny processors** sitting on one die. We call each of those processors a **PE** (short for "Processing Element").

The PEs are arranged in a rectangle of about 750 rows by 990 columns. Each PE is small. It has its own little CPU, around 50 KB of fast on-chip memory, a local scheduler, and four ports that connect it to its four neighbors (north, south, east, west). There is no shared memory. There is no central scheduler. PEs talk to each other only by sending little messages out one port and receiving them at a neighbor's port.

A useful mental picture is a giant city grid of 900,000 small houses. Each house has four doors, one on each side. Each door opens onto a short alley that leads to the next-door house. Every house can talk to every other house, but only by passing notes hand-to-hand down those alleys.

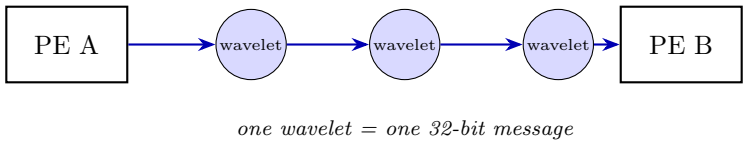


The "fabric" is the name we give to all the wires and routers connecting those PE ports together. When a PE sends a message, the fabric carries it along to wherever it needs to go.

1.2 What is a "wavelet"?

A **wavelet** is the unit of message that travels on the fabric. It is just one 32-bit number. That is all. The PE puts a 32-bit value into one of its output ports, and the fabric carries that value, hop by hop, to wherever its route says it should go. Each hop takes a fraction of a nanosecond. By the time the wavelet arrives at the destination PE, that PE's CPU can pick it up out of an inbox, look at the bits, and decide what to do.

Everything that happens on WSE-3 is built out of wavelets going back and forth. PEs emit wavelets, PEs receive wavelets, PEs do work in between. There is nothing else on the wires.



1.3 Why do we need "colors"?

A **color** is a label stamped on every wavelet. The fabric's routers look at that label to decide where the wavelet should go next. You can think of a color like a channel on a walkie-talkie. If two people are tuned to channel 5, they hear each other. If one is on channel 5 and one is on channel 7, they hear nothing from each other.

Why do we need more than one color? Because at any given moment, a lot of different conversations are happening on the same chip in parallel. For example, in our graph-coloring kernel:

- Row 3 of PEs might be passing graph-color updates from west to east.
- Row 4 might be running a global "are we done?" check from east back to west.
- Column 2 might be broadcasting a synchronization bit from north to south.

All three things have to happen at the same time, on the same fabric. If they all used the same color, the routers would have no way to tell them apart. A wavelet might end up going the wrong direction, or two wavelets might collide on the same wire. By giving each separate stream its own

color, and by setting up the routers ahead of time to know "color 5 always goes west, color 7 always goes east," we keep the streams separate.

We use about ten colors in our kernel. WSE-3 gives us only 32 colors total to work with. That is the entire palette for wiring the chip, so we have to spend them carefully.

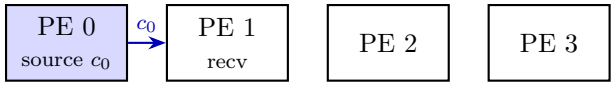
The single-source-per-color rule. There is one more important rule about colors. It comes from the way the fabric's routers are configured. **For any given color, exactly one PE is allowed to originate new wavelets on it.** Every other PE that the color's path runs through is set up either to pass the wavelet along, or to receive the wavelet at its CPU. No other PE may also start a new wavelet on the same color.

Why is that? The routers are configured at compile time, before the program runs. Each router knows ahead of time, for each color, which direction the wavelets come from and which direction they go. If two different PEs in two different places both tried to inject brand new wavelets on the same color, the routers would get a confused picture of the color's flow. The wavelets would collide where the two flows met. Some would get reordered. Some would silently disappear.

In practice this means every color in our kernel has a clear "owner" PE. For example, color `c_3_E` belongs to PE 3, going east. PE 3 is the only PE allowed to put new wavelets on `c_3_E`. PEs further east of PE 3 can only receive on `c_3_E`. They cannot also send on it. If PE 4 wants to send something east, it must use a different color, like `c_4_E`.

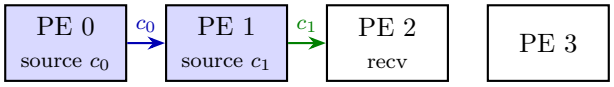
This single-source rule is the reason a naive one-dimensional kernel ends up needing N different colors for N PEs in a row. Each PE needs its own color to be the source. That does not scale. Later in Section 3.4 we will see how we worked around this by inventing **bridges** that let us reuse colors across different parts of the chip.

Single-source rule, in pictures. PE 0 sources color c_0. PE 1 receives on c_0. PE 1 cannot also be a source for c_0.



PE 1 is not allowed to also send on c_0.
If PE 1 wants to send east, it has to use a different color c_1.

PE 0 sources c_0, PE 1 sources c_1. Each PE has its own color.



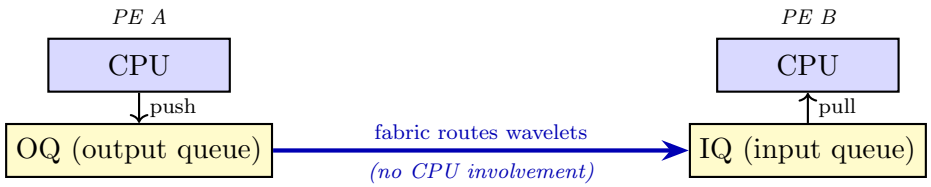
1.4 Why do we need queues?

Wavelets arrive at the CPU very quickly. While a wavelet is landing, the CPU is usually busy doing something else. Maybe it is in the middle of speculating a color for one of its local graph vertices. Maybe it is updating an array in memory. We do not want to drop the incoming wavelet. We also cannot ask the fabric to wait. So we need somewhere to park it.

That parking spot is called a **queue**. It is a small hardware buffer that holds incoming wavelets until the CPU gets to them. Each color the PE wants to receive needs its own input queue (we call

them IQs). Each direction the PE wants to send on needs its own output queue (an OQ).

Queues do another important job. They let the sender and the receiver work at different speeds. Without a queue, every send would be like yelling across a room and waiting for the other person to yell back before you can say anything else. With a queue, the sender just drops the message in the outbox and goes on with its work. The receiver picks the message up whenever it is free. This is what makes the fabric a real pipeline. Many thousands of wavelets can be in flight at once without anyone having to wait.



1.5 Why are queues so limited?

Every queue is real silicon. It needs a piece of memory to hold wavelets, a scheduler that decides when to fire the CPU task, and a slot in the chip's task-ID table. With about 900,000 PEs on one die, every byte of per-PE storage adds up to many megabytes across the whole chip. So Cerebras was very stingy.

Each PE gets exactly six input queues and six output queues that we are allowed to use. (There are two more of each that the system reserves for talking to the host computer.) Six is not a lot.

You can picture it as having only six mailboxes per house in that 900,000-house city. Suppose our program needs to handle eight different kinds of conversation per PE. For example: row data going east, column data going south, row reduce going west, column reduce going north, row broadcast, column broadcast, the back-channel, and so on. We have eight kinds of mail but only six mailboxes. Something has to give.

Fitting our program into six mailboxes per PE has been the single biggest engineering challenge of this whole project. Almost every implementation step described later in this document is some new trick for collapsing eight kinds of mail into six mailboxes.

1.6 Why are colors also limited?

Colors are cheaper than queues, but they are still in short supply. WSE-3 gives us 32 colors total and the system reserves some, leaving us about 24 we can use. That sounds like a lot until you remember the single-source rule. If we tried to give each PE in a row its own color (so that each PE can be the source of its own east-going stream), then a 990-column row would need 990 colors. We only have 24. We are off by a factor of 40.

The way out is **color reuse**. Two wavelets on the same color do not collide as long as their paths through the chip never overlap. So if we can make a color "live" only in one small neighborhood of the chip, and have it "die" before it would collide with the same color used somewhere else, then the router never sees a conflict.

This is the idea behind the segment-and-bridge design described later in Section 3.4. We pick a small number, like four PEs, and let the source colors live only inside groups of four. At the

boundary of each group we have a special PE called a bridge. The bridge consumes the color, runs everything it received through its CPU, and starts a fresh stream on a new color. The next group then reuses the same source color from scratch. The router never sees two simultaneous sources on the same color because they are separated in space by the bridge.

1.7 How does our algorithm sit on this?

Picasso's *speculative graph coloring* is a **BSP loop** — Bulk Synchronous Parallel: every PE runs in lock-step rounds with a global synchronization barrier between rounds. Each round is

> *guess a color, tell neighbors, detect conflicts, revert losers, repeat until stable.*

Each round is a wavelet exchange followed by a **reduce** then a **broadcast** (terms defined below).

Reduce. "Reduce" is the parallel-computing word for *combining many values into one*. Example: every PE has a flag "do I still have an uncolored vertex? (yes/no)." The reduce step asks the global question *"is anyone still uncolored?"* by OR'ing every PE's flag into a single bit. The OR is the "reduction operator", sum and max are other common reductions. On a fabric, a reduce is implemented as a chain of wavelets: each PE receives the partial OR from its neighbor, OR's in its own bit, and forwards the result onward.

Broadcast. Once one PE knows the global answer ("yes, run another round"), it has to **tell every other PE**. That is a broadcast, the same value sent to many recipients. On a fabric it is the reverse direction of the reduce: one wavelet starts at one PE and propagates outward.

Mapped onto WSE-3:

- **Guess:** PE runs locally, no fabric traffic.
- **Tell neighbors:** one wavelet per boundary edge on a color that routes toward that neighbor.
- **Detect + revert:** PE runs locally again.
- **Reduce + broadcast:** a short chain of wavelets along the row (and column, in 2D), accumulating at each PE for the reduce, then a second chain in the reverse direction for the broadcast.

The kernel's job is to make the "tell neighbors" + "reduce + broadcast" traffic fit in the 6-queue / $\sim$10-color budget at any grid size we pick. **That is the engineering problem the rest of this document spells out.**

1.8 Glossary

Quick definitions for the recurring jargon, every time one of these words appears later in the document, it means what it says here.

Term	Plain meaning
PE	"Processing Element", one of the $\sim$900,000 small CPUs on a WSE-3 die.
Wavelet	A 32-bit message sent from one PE to a neighbor over the fabric. The unit of fabric traffic.

Term	Plain meaning
Fabric	The on-die mesh of routers (one per PE) that delivers wavelets between PEs.
Color	A label stamped on a wavelet that tells the routers how to route it. Like a walkie-talkie channel.
Queue (IQ / OQ)	Small hardware buffer that holds incoming (IQ) or outgoing (OQ) wavelets while the CPU is busy. WSE-3 has 6 user IQs and 6 user OQs per PE.
Route	The per-PE configuration that says "for color X, accept wavelets from direction Y and send them out direction(s) Z." Configured by the layout file at compile time.
RAMP	A special "direction" name for the PE's own CPU. `rx=RAMP` means the CPU originates wavelets on this color; `tx=RAMP` means deliver these wavelets to the CPU.
BSP	Bulk Synchronous Parallel, every PE runs a round of work, syncs at a barrier, runs the next round, etc.
Round	One pass of the BSP loop. Within a level, Picasso may take several rounds to resolve all conflicts.
Level	One Picasso recursion stage. Each level uses a smaller palette and processes the vertices that remain uncolored after previous levels.
Reduce	A collective that combines a value from every PE into one global value (e.g. OR of all "uncolored?" flags). Implemented as a wavelet chain.
Broadcast (bcast)	A collective that distributes one PE's value to all PEs. Reverse direction of a reduce.
Boundary	A vertex on PE A whose graph neighbor lives on PE B. Boundaries are the only vertices whose color updates need to leave the PE.
Speculative coloring	The PE picks a color guess locally, sends to neighbors, and finds out later whether it conflicts. The "loser" reverts and re-guesses next round.
Path C	Our partitioning rule: assign vertices to PEs by contiguous GID ranges in row-major order so that "lower-GID wins" automatically becomes "western (or northern) PE wins."
GID	Global vertex ID. A unique integer assigned to every graph vertex.
Segment	A run of S consecutive PEs along an axis that share a small set of source colors. Segments let us reuse colors instead of allocating one per PE.
Bridge	The last PE of a segment. Receives wavelets on the in-segment colors, re-emits them on a *bridge color* so the next segment can decode the merged stream.
Bridge color reuse	Bridge color c_{be} is alive only between bridge k and bridge $k{+}1$. Bridge $k{+}2$ can reuse c_{be} because the previous use has terminated.

Term	Plain meaning
Back-channel	A wavelet path that flows *opposite* to the main data direction (here, westward through a row whose data flows east). Needed for "south-west" anti-diagonal cross-PE conflicts where the winner sits east of the loser.
e2s relay	"East to South" relay. A PE that re-emits east-arriving wavelets onto the south-going stream so columns east of the sender also receive them.
In-band opcode	A scheme where a few bits of the 32-bit wavelet are reserved as a "type tag" so multiple logical streams (data, data-done, broadcast, reduce) can share one fabric color. The CPU dispatches on the bit pattern.
LWW	"Last Writer Wins", the conflict-resolution rule. When two PEs claim the same color for adjacent boundary vertices, the higher-GID PE reverts. Today this is just "lower-GID wins" plus speculative guessing.
Pipelined-LWW	Our family of fabric-pipelined kernels (as opposed to `sw-relay` which involves the host between every hop).
Data-done sentinel	A special wavelet (one per PE per round) that says "I have finished sending all my data wavelets for this round." Receivers count these to know when it is safe to detect conflicts.
Round-parity tag	One bit (`[30]=parity`) on every wavelet that says "round even" or "round odd." Lets a fast PE send round-k+1 wavelets before a slow PE finishes round-k without confusing the receiver.
OQ backpressure	When the OQ is full, the CPU's next `@mov32` send call blocks until the fabric drains. Not a bug, but a perf cliff under heavy traffic.

2 Summary: which implementations scale to WSE-3?

We maintain three parallel kernel families under `picasso/run_csl_tests.py`.

Routing / layout	Files	Dim support	Scales to WSE-3?
`sw-relay` (baseline)	`csl/layout.csl`, `csl/pe_program.csl`	1D and 2D any N	**Yes (trivially)**, every PE is independent, no color/queue pressure. Slow but correct.
`hw-filter`	`csl/layout_hw_broadcast.csl` et al.	1D only	No, 2D would need multicast switches we do not use. Out of scope.
`pipelined-lww`	family below	varies	**Yes, at $S = 2$ dual-axis** (`2d_seg2`).

2.1 Pipelined-LWW sub-variants

`--lww-layout`	Dim support	Validated at	Scales	Notes
`bidir`	1D, ≤ 5 PEs	1×4	No	Bidirectional east+west; killed by 6-queue cap beyond 5 PEs + CP3 reinject rule.
`east`	1D, ≤ 5 PEs	1×4	No	Single-segment east-only.
`east_seg`	1D, any width	1×16	**Yes**	Segmented east-only with bridge-color alternation. Queue use constant per PE.
`2d`	2×2 only	2×2	No	Iter-2 fixed plumbing.
`2d_seg`	single-axis	1×16, 16×1	**Yes**	Axis-agnostic segmented east/south.
`2d_seg2`	dual-axis $2 \times 2 \ldots 16 \times 16$	through 16×16 sparse	**Yes**	$S = 2$ segments, merged back-channel on reduce chain, in-band row/col bcast.
`2d_multicast`	2D small grids	4×4 probe	Partial	Multicast switches; 4-way fanout capped at 5×5. Not the chosen path.

Short answer. For WSE-3, `--lww-layout 2d_seg2` is the design that actually closes the queue, color, and route budget at $S = 2$ with a *grid-size-invariant* kernel. The same compiled binary runs at 2×2, 4×4, 8×8, 16×16, and 2×16 today; larger sizes are simulator- bound but architecturally identical.

3 The implementation story, how we got here, step by step

This section walks through every kernel we have built, in the order we built them. Each one was written **to solve a specific limitation of the one before it**. By the end you should see exactly *why* the design ended up the way it did. none of the choices were arbitrary; each one came from running into a wall with the previous design and looking for the smallest change that would let us climb over it.

The walls we kept hitting are always one (or two) of:

1. **Six queues per PE.** Every distinct color we want the CPU to receive on, or send on, costs one queue. If we end up wanting eight kinds of conversation per PE, two of them have to share somehow.

2. **One source per color.** A color belongs to exactly one PE in any given route. We cannot just "add another sender", we have to invent a new color, or have the CPU re-emit on a different color (a *bridge*).

3. **One rx-direction per color.** For a single color at a single PE, the route accepts wavelets from at most one side (`EAST`, `WEST`, `NORTH`, `SOUTH`, or `RAMP`). This is what forces *alternating* color chains for any operation where the CPU has to receive *and* re-send on the same logical stream.

4. **Total color budget ~ 24.** Once we run out of colors, the only way forward is to reuse them on disjoint regions of the grid.

Keep these four walls in mind as you read, every jump from one implementation to the next was a way around one of them.

3.1 `sw-relay`, start with something that just works

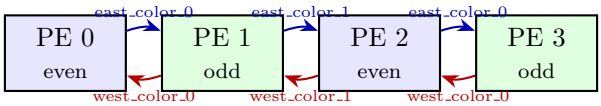
Files: `csl/layout.csl`, `csl/pe_program.csl`.

The first thing we needed was a kernel that produced *correct* colorings at any grid size. Speed could come later. So `sw-relay` does the simplest thing that works: each PE owns its local vertices, and when a vertex on PE A needs to know about a boundary neighbor on PE B, the wavelet travels one hop on the fabric, and the host (or a small CPU task) reads it and decides what to do next. The "SW" in `sw-relay` means **software relay**: multi-hop traffic is not pipelined on the fabric, every hop involves a software step.

For colors and queues this kernel is generous. It uses an eight-color checkerboard for east/west-/north/south data (alternating even/odd colors by PE parity so adjacent PEs do not try to send and receive on the same color at the same moment). Three more colors handle the global synchronization barrier. Per-PE we use about four data queues plus one or two for the barrier. We are nowhere near the 6-queue cap.

The reason `sw-relay` is still in the repo is that it works at *any* grid size and *any* aspect ratio, so it remains our correctness oracle: when a new pipelined kernel produces a suspicious coloring, we run the same graph through `sw-relay` and diff. The reason it is **not** the scaling target is exactly that "every hop involves software." On big fabrics with many hops, the software step becomes the bottleneck. We want the wavelets to fly across the fabric without CPUs in the loop. That want is what motivates everything else.

Layout sketch (1D, 4 PEs). Eight data colors form a checkerboard. Adjacent PEs use opposite-parity colors so a sender and a receiver are never trying to use the same color in the same instant. Three more colors carry the global synchronization barrier (not drawn).



Per-PE queue use (any interior PE).

Queue	Purpose
2 IQ (east + west data)	receive from both neighbors
2 OQ (east + west data)	send to both neighbors
1 IQ (barrier reduce)	global sync, in-bound
1 OQ (barrier broadcast)	global sync, out-bound
total	6 queues, well inside the 6-queue cap

The CPU does software relay: when it receives a wavelet, it inspects the sender ID and decides whether to forward the wavelet to the next hop. The host is in the loop on every hop, which is why `sw-relay` is correct but slow.

3.2 `pipelined-lww bidir`, the first attempt at a fabric pipeline

Files: `csl/layout_lww.csl`, `csl/pe_program_lww.csl`.

So we asked: can each PE just *broadcast* its colored vertices into the fabric, in both directions, and let the receiving PEs filter for the wavelets they care about? That is what `bidir` is. Every PE k owns **its own east-going color** `c_k_E` and **its own west-going color** `c_k_W`. PE k injects every newly colored boundary vertex onto both colors. PE $k{+}1$ receives PE k's east color from its west door, scans its boundary list, and keeps only the wavelets whose sender ID matches a graph neighbor it cares about. PE $k{-}1$ does the same with PE k's west color.

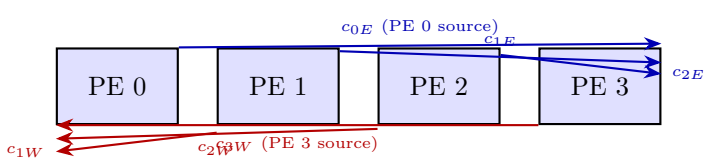
This is the first place we ran into the **single-source-per-color rule** in practice. We could not just say "everyone sends on `c_E`" and let the fabric figure it out, the routes have to be preconfigured to know exactly which PE originates each color, so there must be one color per source PE. That immediately means **each PE consumes one color slot per direction** ($2N$ colors total for an N-PE row), and **each interior PE needs one input queue per remote sender it might hear from** ($N{-}1$ for east-plus-west neighbors combined, in the worst case).

For 4 PEs in a row that is 8 data colors plus 3 barrier. fine. For 5 PEs it is $10 + 3 = 13$, still under the $\sim$24 usable colors but starting to pinch. For 6 PEs an interior PE needs 5 input queues just for data plus 2 for the barrier. already 7, *one over* the 6-queue cap.

So `bidir` worked beautifully up to 4–5 PEs in a row and proved that the fabric pipeline is genuinely faster than `sw-relay` (1.1×–1.4× speedup on tests 1–10, recorded in Section 7.1). It also introduced a piece of machinery we kept ever after: the **round-parity tag**, one bit on every wavelet that says "I belong to round k even" or "I belong to round k odd." Without it, a fast PE that finished round k early might emit round-$k{+}1$ wavelets while its slow neighbor was still processing round k; the receiver would mis-attribute the early wavelet and lose a flag. With the parity bit, the receiver can park "wrong-parity" wavelets in a staging buffer until its `reset_round_state` runs.

But `bidir` could not scale past five PEs, and we wanted to color graphs much bigger than that. So we asked the next question.

Layout sketch (1D, 4 PEs). Each PE owns its own east-going color and its own west-going color. Every PE broadcasts on both. Downstream receivers filter by sender ID at the CPU.



Per-PE queue use (interior PE 1 of 4).

Queue	Bound to
1 OQ	own east-going color c_{1E}
1 OQ	own west-going color c_{1W}
1 IQ	receive from PE 0 east color c_{0E}
1 IQ	receive from PE 2 west color c_{2W}
1 IQ	receive from PE 3 west color c_{3W}
1 IQ + 1 OQ	barrier reduce / broadcast
total	4 IQs + 3 OQs

For 5 PEs in a row this just fits. For 6 PEs it no longer fits: the interior PE would need 5 east+west IQs, and we are out of queue slots.

3.3 `pipelined-lww east`, half the colors by exploiting Path C

Files: `csl/layout_lww_east.csl`, `csl/pe_program_lww_east.csl`.

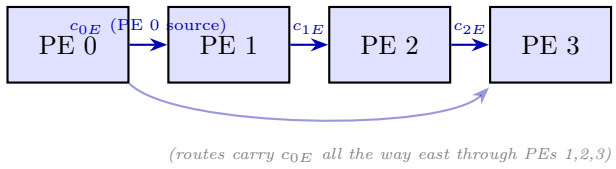
Why are we sending wavelets *both* directions when the conflict rule already says "higher-GID always loses"? If we assign vertex IDs in row-major order (PE 0 owns the lowest IDs, PE 1 the next chunk, ..., PE $N-1$ the highest), then for any cross-PE conflict, the eastern PE has the higher ID and will be the one to revert. The eastern PE *needs* to hear from the western PE in order to detect the conflict. The western PE does **not** need to hear from the eastern PE, it cannot lose, so it has nothing to do with the news.

We named this the **Path C invariant**. *contiguous monotone GID allocation guarantees the western PE always wins.* Once we adopted Path C in the host partitioner, the entire westbound half of `bidir` became dead weight. Dropping it gave us `pipelined-lww east`: same per-PE color scheme as `bidir`, but only the east-going half. The color count drops from $2N$ to N; the queue count drops by half on the data side.

Does this scale further? Not really. With $N-1$ remote east-senders to listen to, an interior PE still needs $N-1$ input queues just for east data, and the barrier still needs two more. At 6 PEs you are already at 7 IQs. The cap shifted from "5 PEs" to "5 PEs with a little more headroom". fundamentally the same wall.

But what `east` taught us was very valuable: **the partitioning choice on the host can collapse half the fabric traffic on the device.** That was the door to color-reuse. If we are no longer trying to hear from every other PE individually, maybe we do not need a separate color per sender at all.

Layout sketch (1D, 4 PEs). Same as `bidir` but with the entire westbound half deleted. Each PE owns one east-going color. Path C on the host guarantees the eastern PE in any conflict is the loser, so westbound data is provably redundant.



Per-PE queue use (interior PE 1 of 4).

Queue	Bound to
1 OQ	own east-going color c_{1E}
1 IQ	receive from PE 0 east color c_{0E}
1 IQ + 1 OQ	barrier reduce / broadcast
total	2 IQs + 2 OQs

Half the queues of `bidir`, but per-PE queues still grow linearly with N. At 6 PEs an interior PE needs 5 east IQs, which puts us back over the queue cap.

3.4 `pipelined-lww east_seg`. Break the per-PE color tax with bridges

Files: `csl/layout_lww_east_seg.csl`, `csl/pe_program_lww_east_seg.csl`.

This is the design that finally scales 1D to any number of PEs. It is built on one big idea: **reuse colors across the chip by letting them die at well-chosen points and starting fresh ones**.

Here is how it works. We pick a small number called the **segment size**, written S. In our 1D kernel we use $S = 4$. In the 2D kernel we use $S = 2$. Then we chop the row of PEs into segments, each of which contains S PEs in a row.

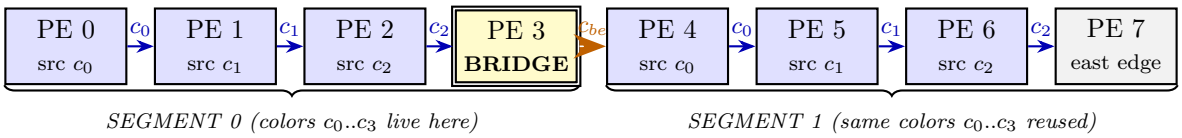
Inside one segment, the PE at the first position injects new wavelets on color c_0. The PE at the second position injects on c_1. The third uses c_2. The fourth uses c_3. The downstream PEs in the same segment receive on those colors. So far this is just like the `east` kernel, except that we only need S colors instead of N.

The trick is the **bridge**. The last PE of every non-final segment is treated specially. It does not inject anything new on its own source color toward the next segment. Instead, its CPU listens on every color that flowed into the segment, and for each wavelet it receives it re-emits a copy on a new color called the **bridge color**. The bridge color is shared across the rest of the chip according to a clever alternation: the first bridge uses `c_be`, the second uses `c_bo`, the third uses `c_be` again, the fourth uses `c_bo` again, and so on. We alternate between just two bridge colors no matter how many segments there are.

Once the bridge has consumed everything from segment 0 and re-emitted on `c_be`, the source colors c_0, c_1, c_2, c_3 are no longer "alive" anywhere east of the bridge. So the next segment is free to reuse those same source colors from scratch. The router never sees a collision because the two uses of c_0 are separated by the bridge.

The bridge colors do the same trick one level up. `c_be` is alive only between bridge k and bridge $k{+}1$. At bridge $k{+}1$ the CPU consumes `c_be` and starts a new flow on `c_bo`. So bridge $k{+}2$ can reuse `c_be` again without conflict. Two bridge colors alternating in this pattern is enough for any number of segments.

A picture might help. Here is a row of 8 PEs split into two segments of 4 with one bridge between them:



Reading the diagram. Source colors c_0, c_1, c_2 fan out from each PE to its right-hand neighbor inside one segment. They die at the bridge (PE 3), which reads every incoming wavelet on its CPU

and re-emits it on the bridge color c_{be}. From PE 4 onward the same source colors c_0, c_1, c_2 start fresh because the previous uses are gone. The router never sees two simultaneous sources of c_0 because the bridge acts as a clean cut between segments.

The numbers come out cleanly. We use S source colors plus two bridge colors plus three barrier colors. That is **nine colors total no matter how wide the row is**. The per-PE queue count is also constant. An interior bridge PE needs two output queues (one for east data, one for the reduce chain). It also needs six input queues (the reduce receiver, the broadcast receiver, and four data-slot receivers that cover the bridge color plus the in-segment source colors). That fills exactly six out of six input queues at $S = 4$. We cannot grow S any further in this kernel without breaking the queue cap, but we can grow N as much as we want by adding more segments. The total chip width becomes unbounded.

This was the first kernel that ran a 1×16 row from end to end. It also locked in the **segment plus bridge** pattern that every later 2D kernel inherits. We did pay one cost. The bridge PE has to do CPU work for every wavelet that crosses it. The CPU receives the wavelet on one color, writes it briefly to memory, and emits it on a different color. That is slower than a pure fabric forward where the wavelet would just sail through the routers without any CPU involvement. But CPU re-emission is the price we pay to dodge the single-source rule with a finite color budget.

Per-PE queue use. Per-PE queue counts now stay constant regardless of N.

PE position	Queues used
in-segment (non-bridge)	2 IQ data + 1 IQ red + 1 IQ bcast + 1 OQ east + 1 OQ red
bridge PE	4 IQ data ($S-1$ in-seg + bridge color)+ 1 IQ red + 1 IQ bcast + 1 OQ east (bridge color) + 1 OQ red
west edge (PE 0)	1 OQ east + 1 OQ bcast + 1 IQ red
east edge (last PE)	data IQs only, no east OQ

Color budget: S source colors + 2 bridge colors + 3 barrier colors = 9 colors total, independent of N.

What `east_seg` cannot do is 2D. The row direction is solved. The column direction is untouched. So we asked the next question.

3.5 `pipelined-lww 2d` (iter 2), proving the dual-axis idea on a 2×2 toy

Files: `csl/layout_lww_2d.csl`, `csl/pe_program_lww_2d.csl`.

Going from 1D to 2D was not just "replicate `east_seg` per row and per column." There were genuinely new problems we did not have in 1D, and `2d` (in its second iteration. the first one falsified itself) was where we figured them out, on the smallest possible grid: 2×2. That meant only four PEs, but every new transport idea we needed lived inside that toy.

The new problems:

(a) Anti-diagonal cross-PE conflicts. Under Path C, when a boundary edge crosses *both* a row boundary and a column boundary at once, the loser sits to the **south-west** of the winner. East data does not reach SW; south data does not reach SW; we genuinely need a westbound stream. We added `c_W_data_r1`, a row-1 westbound color, that PE(1,1) uses to forward south-arriving wavelets back to PE(1,0). That is the **back-channel** pattern, and we have used it ever since.

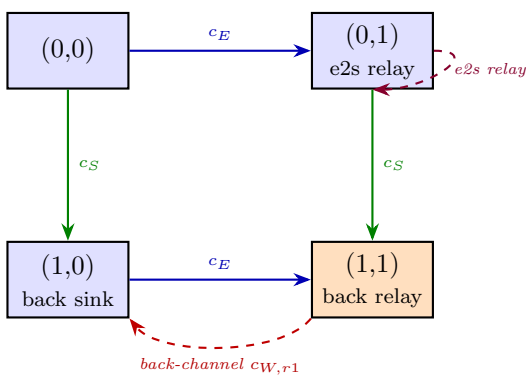
(b) Origin pixel reaching all rows on its column. When PE(0,1) publishes a color, it needs to reach not just east-of-row-0 but also any south PE on column 1. We added the **e2s relay** ("east to south"): at row-0 PEs that are not col 0, every east arrival gets re-emitted south. So PE(0,0)'s east wavelet reaches PE(0,1), which forwards it onto col 1 going south, reaching PE(1,1).

(c) Running out of fabric colors. Adding the new back-channel and the second-axis colors blew our budget. So we invented the **in-band opcode trick** for the first time: instead of giving the row broadcast its own dedicated color (which would cost one queue at every receiver), we packed a single bit into the high end of the data wavelet, bit 29. to mark it as "this is a broadcast, not a data wavelet." The receiver task in the kernel just looks at bit 29 and dispatches accordingly. One color now carries two logical streams.

(d) Multi-PE OR-reduce on each axis. The 1D kernel already had alternating reduce chains; in 2D we need them on both rows and columns. We replicated the same pattern (`c_row_red_c0/c1`, `c_col_red_c0/c1`).

`2d` iter 2 made all of this fit in 6 queues per PE, but only for the 2×2 shape. Several things were hardcoded: the back-channel was attached to row 1 specifically, the e2s relay was at row 0 specifically, and the col-broadcast went down a single column. The kernel had no idea how to be told "okay, now you are 4×4." We needed something more general.

Layout sketch (2×2). The toy grid where every 2D primitive was invented. East data on c_E, south data on c_S, row-1 westbound back-channel on $c_{W,r1}$, row-0 east-to-south relay re-emits east arrivals onto c_S, and row/col reduce chains use alternating colors.



Per-PE queues (2×2). Fits 6/6 by hardcoding the topology to 2×2:

PE (0,0) NW	1 OQ east + 1 OQ south + barrier IQ/OQ row + barrier IQ/OQ col
PE (0,1) NE	1 IQ east (recv c_E) + 1 OQ south (e2s relay) + barriers
PE (1,0) SW	1 IQ south + 1 IQ back-channel + 1 IQ east + barriers
PE (1,1) SE	1 IQ east + 1 IQ south + 1 OQ back-channel + barriers

But before going general, we made one detour first.

3.6 `pipelined-lww 2d_seg`, single-axis segmented in 2D namespace

Files: `csl/layout_lww_2d_seg.csl`, `csl/pe_program_lww_2d_seg.csl`.

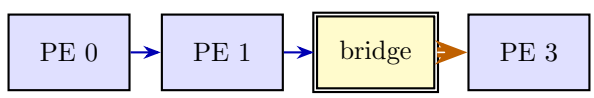
While we were figuring out the right way to do dual-axis, we needed a kernel that could at least scale 1D *or* a single column to many PEs in the 2D framework. `2d_seg` is exactly `east_seg` lifted into the 2D namespace, with a small twist: when the runner asks for $1 \times N$ the segmented pipeline runs east; when it asks for $N \times 1$ the same pipeline runs south. Internally the routes use a direction-agnostic macro pair (`DIR_IN`, `DIR_OUT`) that resolves to `WEST/EAST` or `NORTH/SOUTH` based on which axis is active. The kernel sees the active-axis length as `num_cols` regardless, so all the segment math is the same.

Color and queue counts are identical to `east_seg`. The only new thing this kernel does is *handle either axis with one codebase*. Validated up to 1×16 and 16×1.

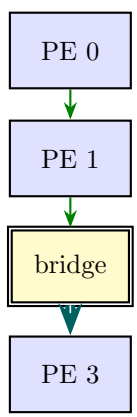
This kernel is still in active use, when someone asks for $1 \times N$ or $N \times 1$ today via `--lww-layout 2d_seg2`, the layout file detects the single-axis case and dispatches *this* kernel under the hood. So `2d_seg` is the stable 1D backbone underneath the dual-axis kernel.

Layout sketch. Same colors and queues as `east_seg` but the layout file picks the active axis at compile time. When `num_rows == 1`, the segmented pipeline runs east. When `num_cols == 1`, the same pipeline rotates ninety degrees and runs south.

1×N mode (east)



N×1 mode (south)



Per-PE queues. Identical to `east_seg`: 9 colors total, 6 queues used at the bridge PE, fewer elsewhere. The runner enforces that exactly one of `num_rows` or `num_cols` equals 1 for this layout.

3.7 `pipelined-lww 2d_seg2`. Composing everything for any grid

Files: `csl/layout_lww_2d_seg2.csl`, `csl/pe_program_lww_2d_seg2.csl`.

This is the kernel where everything we learned came together. The goal was simple in words. **Run dual-axis at any grid size from 2 by 2 to as big as the fabric permits, with one fixed compiled binary.** We pin the segment size at $S = 2$, so each segment is two PEs wide on each axis, and we stack every trick we have learned. There are five pieces:

1. Segment plus bridge on the row axis, exactly as in `east_seg`. Source colors are `c_0` and `c_1`. Bridge colors are `c_be` and `c_bo`.

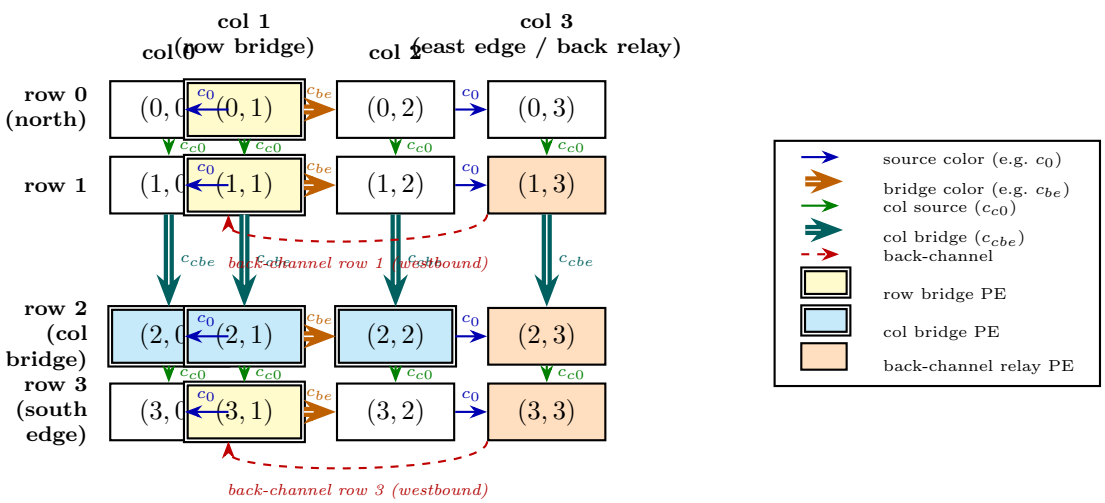
2. Segment plus bridge on the column axis, the mirror image of the row axis. Source colors are `c_col_0` and `c_col_1`. Bridge colors are `c_col_be` and `c_col_bo`.

3. The east-to-south relay, generalized. In the old `2d` kernel only row 0 had this relay. Now any PE with `col > 0` re-emits east arrivals onto its south stream.

4. The per-row back-channel, generalized. In the old `2d` kernel only row 1 had a back-channel. Now every row above row 0 has its own westbound back-channel that runs from the east-edge column toward column 0. The back-channel carries the south-west anti-diagonal wavelets that the rest of the topology cannot reach.

5. The alternating reduce chain on each axis. This is the same `sync_reduce_c0` and `sync_reduce_c1` pattern we have used since the 1D kernel, replicated on rows and columns.

Here is what the layout looks like at 4 by 4:



Things to notice in that picture:

- The row of PEs across the top is just like our 1D `east_seg` kernel, with source colors and a bridge column at position 1.

- The same pattern repeats vertically. Columns have their own source colors `c_col_0` and `c_col_1` and their own bridge colors `c_col_be` and `c_col_bo`. Row 2 in this picture is a column-bridge row.

- For every row above row 0, the rightmost PE (the back relay) sends a westbound back-channel toward column 0. The back-channel carries the wavelets that originated north and east of the receiver but need to reach a south-west receiver.

Putting all five pieces together gives us a working dual-axis kernel. Unfortunately it also blows the 6-queue budget at the hardest-working PEs. At a fully interior bridge PE (one that is a bridge on both axes) we counted seven or eight distinct input queues needed. We had to fix that. We did it in a sequence of three checkpoints that we named CP2d.c, CP2d.d.1, and CP2d.d.2. Each one collapsed one stream into another by adding a few opcode bits to the wavelet header.

CP2d.c. We moved the column broadcast in-band onto the south data stream. We used a bit-29 opcode flag, exactly the same trick the old `2d` kernel had used for the row broadcast on the east data stream. Now there is no separate `col_bcast_recv` input queue. The south data input queue handles both data wavelets and broadcast wavelets, and the receiver task tells them apart by looking at bit 29. We freed one input queue.

CP2d.d.1. We did the same thing for the row broadcast on the east data stream. One more input queue freed. We also got back one output queue (the dedicated row broadcast OQ).

CP2d.d.2. This one was the real breakthrough. We folded the entire back-channel onto the existing row reduce chain. Before this checkpoint, the back-channel had its own dedicated color and its own input queue at every interior PE. The reduce chain already had exactly the right shape for what the back-channel needed (an east-edge initiator, alternating colors through the row, a west-edge consumer). So we let back-channel data wavelets ride the same colors as the reduce wavelets, distinguished by a second opcode bit, bit 28. The receiver task at each chain PE looks at bit 28. If it is a reduce wavelet, the task does the OR-and-forward step. If it is a back-channel data wavelet, the task hands it to the south-data handler and re-emits it westward on the same chain.

After CP2d.d.2 the interior-interior PE queue map locks at six out of six **for any grid size at** $S = 2$. The exact mapping looks like this:

Q2	`reduce_recv_iq` row reduce + back-channel (opcode)
Q3	`south_slot1` col data slot 1 (when needed)
Q4	`rx_iq_0` row data slot 0
Q5	`rx_iq_1` OR `col_reduce_recv` (per-PE choice)
Q6	`col_reduce_recv_iq` (when rx_slot_count > 1)
Q7	`south_rx_iq_0` col data slot 0

Six input queues, all in use, no kind of mail dropped. The same compiled binary now runs at 2 by 2, 4 by 4, 8 by 8, 16 by 16, and 2 by 16. We have validated every one of these grid sizes. Section 7.2 has the cycle counts. From the queue-cap and color-cap point of view there is nothing in the way of the full WSE-3.

For a while we worried that the chain-merged back-channel might run into output-queue backpressure on dense graphs. The concern was that test12 (twenty-node graph at 8 by 8) would push so many wavelets through `tx_reduce_oq` on each chain PE that the CPU's `@mov32` would block waiting for the fabric to drain. We even sketched a follow-up checkpoint, CP2d.e, that would move the back-channel back onto its own dedicated fabric-forwarded color. **In the latest rerun, however, test12 at 8 by 8 PASSED**, with 5 levels and 952,663 cycles total. Whatever the earlier hang was, it appears to have been resolved by the layout fixes that landed alongside CP2d.d.2. We are keeping CP2d.e as a possible future optimization rather than a required scaling fix.

3.8 `pipelined-lww 2d_multicast`, an orthogonal optimization

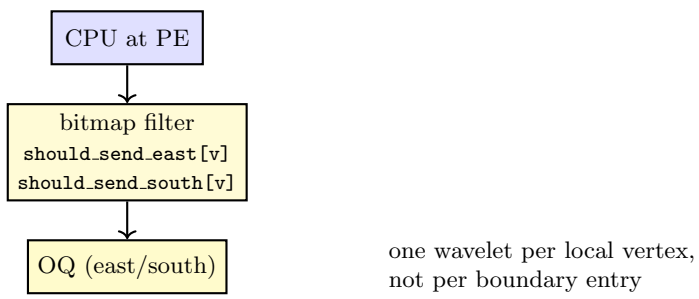
Files: `csl/layout_lww_2d_multicast.csl`, `csl/pe_program_lww_2d_multicast.csl`.

This kernel is not part of the scaling story; it is an *efficiency* sidequest layered on top of `2d` iter 2. The observation: in our other kernels, when a local vertex has, say, three graph neighbors that all happen to live on the same downstream PE, we generate three identical wavelets, one per boundary entry, even though one would have sufficed. That is wasted bandwidth.

`2d_multicast` fixes this by iterating *local vertices* instead of *boundary entries* when packing the send buffer, and consulting two bitmaps uploaded by the host: `should_send_east[v]` and `should_send_south[v]`. If a vertex has no boundary neighbor on the east axis, the kernel just skips its east wavelet. If it has at least one but all on the same downstream PE, one wavelet still goes out, but only one.

This is a real perf win, but it inherits all the 2×2 shape limits of `2d` iter 2, it does not by itself extend to larger grids. The deduplication idea would layer cleanly on top of `2d_seg2` if we ever bring the two together.

Layout sketch. Identical to `2d` iter 2 (same fabric, same colors, same queues). The only change is at the *sender* CPU: it consults a host-uploaded bitmap before emitting each wavelet. Same picture as §3.5 with an extra filter step at every PE's send path.

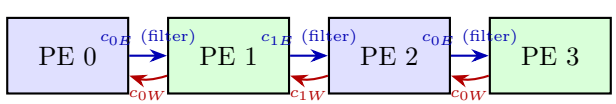


3.9 `hw-filter`, the dead-end branch worth knowing about

Files: `csl/layout_hw_broadcast.csl`, `csl/pe_program_hw_broadcast.csl`.

For completeness: at one point we tried to use the WSE-3 hardware broadcast filter, a feature that lets the fabric drop wavelets whose payload does not match a per-PE filter. The idea was that we could send wavelets from a PE to *all* its downstream neighbors at once and let the hardware filter pick out the right ones. The filter works for one-hop delivery, but multi-hop would have required SW relay buffers we never built, and the cap was around 5 PEs (1D only). Not worth pursuing once `east_seg` was working. We keep the kernel around for reference.

Layout sketch (1D, 4 PEs, HW broadcast filter). Four-color checkerboard, exactly like `sw-relay` except the fabric switches drop wavelets whose payload does not match the receiver's filter. One-hop only.



Per-PE queues. 2 IQ + 2 OQ data + barrier IQ/OQ, inside the cap. The fabric switch's HW filter takes care of dropping non-matching wavelets, so the CPU only ever sees relevant ones.

4 WSE-3 architectural limitations that shaped the design

4.1 Six-queue-per-PE cap (hard)

WSE-3 exposes 6 user input queues and 6 user output queues per PE (queues 2–7; 0–1 are reserved). Every fabric color the CPU receives consumes one IQ; every color it emits consumes one OQ. Multiple colors cannot share a queue.

Impact. A naive 2D kernel needs 2 row-data slots + 2 col-data slots + row-reduce-recv + row-bcast-recv + col-reduce-recv + col-bcast-recv + back-channel-recv $\approx$ 8 IQs. Over-budget.

4.2 Single rx-direction per color (hard on WSE-3)

A fabric color's route at each PE accepts wavelets from *at most one* direction (`RAMP`, `EAST`, `WEST`, `NORTH`, `SOUTH`). `tx` can be multiple. CSL enforces this on WSE-3 with the error `expected at most 1 input direction(s) on 'wse3', got: 2`.

Impact. A single color cannot both accept CPU-injected wavelets (`rx=RAMP`) *and* forward fabric arrivals (`rx=EAST`). Accumulating chains require **two alternating colors per axis** with OR-at-CPU between hops.

4.3 CP3: same-color inject + non-RAMP rx is silently dropped

If a PE's CPU emits on color X while the color's route has `rx` from a fabric direction (not RAMP), the injected wavelet is silently dropped on WSE-3, no compile error, no runtime fault. Forbids fan-in patterns where every PE injects its own bit on a shared color.

4.4 One OQ = one color

Binding an OQ to color A and emitting with `fabric_color =` B is a runtime fault that surfaces as a kernel stall (host-side timeout, `std::runtime_error: received length 0`). Discovered 2026-04-21 in `csl/pe_program_lww_2d.csl` iter 1.

Impact. Two stream types on disjoint colors require two OQs. Combining them in-band on the same color via opcode bits avoids the OQ inflation, this is how we moved `row_bcast`, `col_bcast`, and eventually back-channel data in-band.

4.5 Finite fabric-color budget (~ 24 usable)

WSE-3 has 32 colors total; after `memcpy` reservations and the three fixed barrier colors we target, roughly 24 are usable for data-plane plumbing. A per-PE-dedicated color scheme does not scale.

4.6 OQ depth is small (~ 32 wavelets), backpressures without deadlock

Each OQ buffers a small number of wavelets. Pushing more than it can hold blocks the CPU at `@mov32` until the fabric drains. Under heavy load, a chain where every PE re-emits every wavelet through its own OQ saturates quickly. Not a correctness bug, but a performance cliff (observed at 8×8 test12 under `2d_seg2`, see Section 5.6).

4.7 Monotone ID vs. geometry mismatch

Path C assigns contiguous GID ranges in row-major PE order so the invariant

$$\text{gid}_a < \text{gid}_b \;\Rightarrow\; \text{pe}(\text{gid}_a) \leq \text{pe}(\text{gid}_b)$$

holds. Under "lower-GID wins," the eastern / southern PE always loses. On the col axis (and the anti-diagonal SW class) the loser sits south-west, fabric data flows north-south and west-east, so we reach SW receivers via a *back-channel* ($\text{PE}(R, \text{last_col}) \rightarrow$ westward chain).

5 How we overcame each limitation

5.1 Queue-cap collapse via in-band opcodes

Rather than allocate a distinct color per stream type, we pack stream-discriminator bits into the 32-bit wavelet and dispatch on arrival:

```
Data:          [31]=0, [30]=parity, [29:8]=gid, [7:0]=color
Data-done:     [31]=1, [30]=parity, [29]=0, [28]=0
Row/col bcast: [31]=1, [30]=parity, [29]=1, [0]=value
Row-reduce:    [31]=1, [30]=parity, [29]=0, [28]=1, [0]=value
```

The receiver task (`reduce_recv_task` in `csl/pe_program_lww_2d_seg2.csl`) decodes and dispatches. Each opcode merge frees one IQ + one OQ: CP2d.c folded `col_bcast` onto the south data stream, CP2d.d.1 folded `row_bcast` onto the east data stream, CP2d.d.2 folded back-channel data onto the row-reduce chain.

Interior-interior PE queue map at $S = 2$ after CP2d.d.2:

Q	IQ role
Q2	`reduce_recv_iq` (row_reduce + back-channel via opcode)
Q3	`south_slot1` (conditional on rx_slot_count / cy_is_south / num_cols)
Q4	`rx_iq_0` (row data slot 0)
Q5	`rx_iq_1` OR `col_reduce_recv` OR `south_slot1` (per-PE)
Q6	`col_reduce_recv_iq` (when rx_slot_count > 1)
Q7	`south_rx_iq_0` (col data slot 0)

6/6, N-invariant at $S = 2$.

5.2 Color reuse via bridges (CP2a pattern, generalized to 2D)

Per-PE-dedicated colors would saturate the $\sim$24-color budget at num_cols > 5. We avoid this with **span- disjoint reuse**:

- **Source colors** $c_0 \ldots c_{S-1}$ (and $c_col_0 \ldots c_col_{S-1}$) are local to each segment of length S. PE at local_x $= j$ injects on c_j; downstream PEs in the same segment receive; the color dies at the segment's bridge. Reused in the next segment.
- **Bridge colors** alternate by segment index: c_{be} for seg_idx even, c_{bo} for odd. Bridge k terminates at bridge $k{+}1$, so reuse two segments later is safe.

- **Back-channel / reduce colors** reuse by row parity (*c_W_back_re* even rows, *c_W_back_ro* odd).

Total fabric-color count stays near 10–12 regardless of grid size.

5.3 Single-rx constraint → alternating-chain reduce

Because we cannot do `rx={EAST, RAMP}` on one color, the row_reduce chain uses two colors, alternating per-PE parity:

```
PE(R, k) even col:  recv on sync_reduce_c0 (rx=EAST tx=RAMP)
                    send on sync_reduce_c1 (rx=RAMP tx=WEST)
PE(R, k) odd col:   recv on sync_reduce_c1
                    send on sync_reduce_c0
```

CPU at each interior PE OR's the incoming reduce value with its local `has_uncolored` bit and re-emits westward. Terminus PE$(R, 0)$ broadcasts the result east (row_bcast). The same alternating chain now ferries back-channel data via opcode bits.

5.4 CP3 prohibition → CPU-mediated re-injection

Anywhere we want a wavelet to change color mid-route (segment bridges, e2s anti-diagonal relay), we do it via CPU: `bridge_reinject` and `col_bridge_reinject` in `csl/pe_program_lww_2d_seg2.csl` receive the wavelet at RAMP and re-emit on a different color. Slower than a fabric forward, but legal.

5.5 Anti-diagonal SW delivery → per-row back-channel

Under Path C, some cross-PE conflicts have winner in column C_1 and loser in column $C_2 < C_1$ of a row below. We reach them via:

1. PE(R_w, C_1) sends south on *c_col_∗*.
2. At PE(R_l, C_1), the south stream hits a relay that is also a back-channel source (`is_back_relay`: east-edge column for row > 0).
3. That PE forwards the wavelet westward on the back-channel (folded onto the reduce chain).
4. Every PE between C_1 and C_2 also gets a RAMP copy via the CPU tap, so interior SW receivers see the winner.

5.6 OQ-depth backpressure, acknowledged, CP2d.e pending

Dense graphs at 8×8 trigger $\sim$ row $\cdot$ num_cols back-channel wavelets per round that all route through the CPU-mediated chain on `tx_reduce_oq`. Under saturation we stall. The planned mitigation (CP2d.e, Section 8.3) restores a fabric-forwarded back-channel route on a freed queue, keeping the merge only for row_reduce.

6 How the algorithm fits the hardware

Picasso's level-L loop is a BSP:

1. **Speculate**, each PE picks a tentative color respecting its local and remote colors.
2. **Exchange**, each PE sends tentative colors to boundary neighbors. The pipelined-LWW transport serves this phase: per-boundary east / south / back-channel wavelets.
3. **Detect + resolve**, compare local and received colors at shared boundaries; if they match, the higher-GID PE reverts to -1. Under Path C, always the eastern or southern PE.
4. **Barrier**, global OR-reduce of `has_uncolored`. Round done iff zero; else next round. Row-reduce + col-reduce chains + bcasts implement this.

Key architectural fits:

- **Round-parity tagging** (`[30]=parity`) lets fast PEs send round $k+1$ wavelets before slow PEs finish round k; staging buffers promote next-round wavelets at `reset_round_state`.
- **Bridges and alternating reduce** are both direct consequences of the single-rx constraint.
- **Monotone block partition (Path C)** turns the conflict-loser rule into a fabric-direction rule (east / south loses), which drives the entire east + south + SW-back-channel topology.

The algorithm needed essentially no change; only the transport evolved.

7 Timing data collected so far (simulator, WSE-3 arch)

7.1 Small-grid comparison (1D 4-PE, all 13 tests)

Total cycles across all levels. Source: `LWW_PICASSO_RESULTS.md`.

Test	SW-relay (cyc)	LWW east_seg (cyc)	Speedup
test1	30,801	21,889	1.41×
test2	13,067	10,211	1.28×
test3	39,272	29,547	1.33×
test5	71,747	53,974	1.33×
test6	214,684	179,222	1.20×
test7	360,849	326,376	1.11×
test10	34,365	27,614	1.24×

7.2 Dual-axis 2d_seg2 timings (new, 2026-04-24)

Total cycles across all levels per test. Not strictly comparable across grid sizes because `expected_data_done` grows with N and graphs distribute differently.

Grid	Test	Levels	Cycles	Source
4×4	test1	2	12,176	v4 regression
4×4	test3	2	27,572	v4 regression
4×4	test10	2	33,626	v4 regression
4×4	test11	5	276,621	v4 regression
4×4	test12	5	740,376	v4 regression
4×4	test7	3	220,016	v4 regression
8×8	test1	2	66,635	v7 sparse
8×8	test3	2	72,186	sparse
8×8	test12 (dense, 20 nodes)	5	952,663 (1.12 ms)	rerun PASS
16×16	test1	2	252,317	sparse
2×16	test1	2	41,875	rectangular

Source directories under `runs/local/20260424-2d-seg2-*-cp2dd2*`.

Read-outs. Cycle cost scales roughly linearly with grid diameter (16×16 test1 $\approx 3.8\times$ of 8×8 test1), consistent with the $O(\text{rows} + \text{cols})$ barrier chain for sparse graphs. 2×16 is cheaper than 2×2-denser graphs because test1 has little cross-PE traffic. Dense-graph blow-up at 4×4 (test12: 740k cyc, 5 levels) is real but completes; at 8×8 the same graph saturates the back-channel OQ (pending CP2d.e).

7.3 What we are still missing

- No CS-3 hardware runs for `2d_seg2` past 4×4.
- No timing for 16×16 dense tests (simulator is slow at 64+ PEs with dense graphs).
- No apples-to-apples SW-relay baseline at $\geq 8 \times 8$ for comparison.

8 Plan

8.1 Bigger simulator test cases

Goal: drive 8×8 and 16×16 to 13/13 on reasonable graph sizes and record timing.

1. Unblock 8×8 dense (CP2d.e, Section 8.3). Without this, test11, 12, 7 stall past level 0.
2. Fresh batch runs, isolated per test to dodge the simulator regression-mode flake:

```
for t in $(seq 1 13); do
  python3 picasso/run_csl_tests.py \
    --routing pipelined-lww --lww-layout 2d_seg2 \
    --num-pes 64 --grid-rows 8 --test testN_... \
    --run-id $(date +%Y%m%d)-2d-seg2-8x8-t${t}
done
```

   Same pattern at 16×16. Persist each stdout under `runs/local/` for month-over-month diffs.

3. Extend `LWW_PICASSO_RESULTS.md` with a new section per grid (4×4, 8×8, 16×16) showing total cycles, per-level cycles, rounds per level.

4. Plot rounds × cycles-per-round vs. N to see which axis dominates: more rounds (algorithm-bound) vs. more cycles per round (transport-bound).

8.2 CS-3 hardware runs (real WSE-3)

The appliance path (`neocortex/run_cs3.sh`, `run_cs3_batch.sh`) talks to a real CS-3 through `SdkLauncher`. Same binary as simulator.

1. Small set (test1, test5, test12) at 4×4 through `run_cs3.sh`. Validates the binary on real silicon.
2. If 4×4 is green on HW, scale to 8×8. Simulator's dense-graph cliff at 8×8 may not occur on real HW because OQ drain rate is much higher, a data point we genuinely lack.
3. Compare HW timing to simulator cycle counts.

8.3 CP2d.e, dense-graph scaling fix

Restore a dedicated **fabric-forwarded** back-channel route (CP2d.c-style: `rx=EAST tx={WEST, RAMP}`) on a freed queue (Q6 where `col_reduce_recv` fits on Q5; Q3 otherwise). The fabric switch auto-forwards westward at every interior PE. no CPU `@mov32`, no OQ backpressure. Keep the reduce chain on `sync_reduce_c0/c1` with opcode dispatch for row_reduce only.

Acceptance: 8×8 test12 completes; 4×4 13/13 stays green.

8.4 On-device recursion feasibility

Picasso is inherently recursive: each level prunes colored vertices, then the uncolored subgraph is recolored with a smaller palette. Today the recursion is **host-driven**, the runner hands back per-level colorings, builds the next subgraph, re-invokes the kernel. That is roughly 1 ms host-device RTT per level, dominated by the coloring itself at current test sizes, but potentially a floor at WSE-3 scale.

Which kernels can accommodate on-device recursion?

Kernel	Can loop on device?	Why / obstacles
`sw-relay`	Yes, but pointless	Each PE independent; trivial to loop but no speedup (SW relay is the bottleneck).
`pipelined-lww east_seg`	Yes, with kernel changes	Ring-buffered boundary state, per-level re-entry. Needs on-device palette shrinkage, per-PE live-vertex mask, in-kernel level-done → level-begin transition.
`pipelined-lww 2d_seg2`	Yes, same plan as `east_seg`	Barrier + reduce primitives already exist. Main change: reset `remote_recv_color_next`, data-done next-round counters, and fan-in residuals at level transitions.
`2d_multicast`	Unclear	Fabric-switch-heavy; teardown + re-init between levels may be costly. Not the priority.

The practical blocker is not the kernel, it is the data path for *palette changes*. Each level uses a smaller palette; the kernel currently reads `palette_size` from `runtime_config` set by the host. An on-device loop needs a per-level shrinkage step (or just decrement `palette_size` after every level, the simplest).

Recommendation. Once CP2d.e is green, add a small "level controller" local task to `2d_seg2` that gates between levels: at barrier done, if `remaining > 0`, decrement `palette_size`, call `reset_round_state`, `speculate_task_id`, and loop. Saves one host RTT per level. At 8×8 with roughly 5 rounds / level and 5 levels on test12, that is ~ 25 round-trips removed, potentially meaningful at real scale.

8.5 Diagnose the 4×4 / 2×2 regression-mode flake

The same test sometimes passes in isolation but hangs at level $N \geq 1$ when run as part of `--test-range 1-13`. Single- test `cerebras_host.py` invocations are fresh Python subprocesses, each spinning up its own simulator, so kernel state cannot accumulate. The hang is simulator-process-accumulation or a runner-side race.

1. Reproduce under `strace` to see whether a subprocess is stuck on IPC or not exiting.
2. Check whether repeated `cslc` invocations leak anything in `csl_compiled_out/` that a later test reads.
3. Insert a `sleep 1` between tests and see whether the flake vanishes.

Not blocking, the architecture is proven.

9 Open questions

- $S = 2$ forever, or grow to $S = 4$? $S = 4$ halves segment count (fewer bridges, fewer CPU reinjects) but needs 4 row data slots and pushes the queue budget over 6. Keep $S = 2$ for WSE-3 unless there is a specific perf reason.
- Host-driven vs on-device recursion, see previous section. Host-driven is simpler and debuggable; on-device is faster at large N.
- Partition quality (Step 5): hub clustering under naive block hurts test12 cycles by $\sim 50\%$; a degree-aware within-PE renumbering fixes the imbalance. Host-only change; deferred until 8×8 dense is unblocked.
- When do we actually move to CS-3 hardware? Once 8×8 / 16×16 are green in simulator, there is no more reason to stay local.

10 File and run-artifact index

Primary kernel files (active):

- `csl/pe_program_lww_2d_seg2.csl`, dual-axis kernel, CP2d.d.2
- `csl/layout_lww_2d_seg2.csl`, dual-axis layout
- `csl/pe_program_lww_east_seg.csl`, `csl/layout_lww_east_seg.csl`, 1D stable path

Legacy / experimental:

- `csl/layout_lww_2d.csl`, `pe_program_lww_2d.csl`. 2×2 iter-2 (superseded)
- `csl/layout_lww_2d_seg.csl`, `pe_program_lww_2d_seg.csl` , single-axis seg (still dispatched by `2d_seg2` for $1 \times N$ / $N \times 1$)
- `csl/layout_lww_2d_multicast.csl`, multicast fabric, not the main path

Plan / state docs:

- `AGENTS.md`, repo guide, checkpoint history
- `CURRENT_STATE.md`, latest status
- `LWW_PIPELINE_PLAN.md`, detailed checkpoint plan / diffs
- `LWW_PICASSO_RESULTS.md`, original 1D 4-PE benchmark
- `WSE3_SCALING_ANALYSIS.md`, Markdown twin of this document

Recent run artifacts (CP2d.d validation, 2026-04-24):

- `runs/local/20260424-2d-seg2-4x4-tests1-13-cp2dd2-v4/`. 4×4 13/13 PASS
- `runs/local/20260424-2d-seg2-8x8-test1-cp2dd2/`. 8×8 sparse PASS
- `runs/local/20260424-2d-seg2-8x8-test3-cp2dd2/`. 8×8 anti-diagonal PASS
- `runs/local/20260424-2d-seg2-16x16-test1-cp2dd2/`. 16×16 sparse PASS
- `runs/local/20260424-2d-seg2-2x16-test1-cp2dd2/`. rectangular PASS